

UNIT I: Introduction to Deep Learning

Deep Learning: a subset of ML that uses multi-layered ANN to model and understand complex (non-linear relationships) patterns in data. It enables computers to learn hierarchical representations, automatically extracting features from raw input such as images, text, speech, and time series through several layers of abstraction using techniques like **back propagation**. It has led to significant improvements in tasks like image recognition, natural language processing, and reinforcement learning.

Aspect	Human Brain	Neural Network
Basic Unit	Neuron (biological)	Artificial Neuron
Structure	~86 billion neurons interconnected	Layers of artificial neurons
Signal Transmission	Uses electrochemical signals across synapses	Uses numerical values and weights across nodes
Learning Mechanism	through synaptic plasticity and neurochemical changes	through adjusting weights using algorithms like backpropagation
Processing	Highly parallel and distributed	Simulated parallelism using hardware (GPUs/TPUs)
Adaptability	Highly adaptable and generalizes well in unseen situations	Needs large data and training; generalization is limited by design
Energy Efficiency	Very efficient (~20W for the entire brain)	High power consumption, especially during training

types of data handled by deep learning models

- 1. Time Series Data:** Data collected sequentially over time. **Examples:** Stock prices, weather data, ECG signals, sensor readings. **Model Used:** RNNs, LSTM, GRUs. **Use Case:** Forecasting future values (e.g., predicting next day's temperature or stock prices).
- 2. Image Data:** Data in the form of pixel arrays, usually 2D for grayscale or 3D for RGB images. **Examples:** Photographs, X-rays, satellite imagery. **Model Used:** CNNs. **Use Case:** Object detection, facial recognition, medical imaging diagnostics.
- 3. Video Data:** Sequence of image frames over time. **Examples:** Surveillance videos, YouTube clips, autonomous driving footage. **Model Used:** 3D CNNs, CNN + RNN hybrid models. **Use Case:** Activity recognition, video summarization, gesture detection.

common architectural principles followed in designing deep learning models

- 1. Layered Structure (Input–Hidden–Output):** models are composed of layers of interconnected neurons.
 - **input layer** receives raw.
 - One or more **hidden layers** perform feature extraction and transformations.
 - The **output layer** generates final predictions.
- 2. Depth of network** - number of hidden layers. **Width of network** - number of neurons in each layer.
- 3. Activation Functions:** introduce **non-linearity**, allow model to learn complex mappings.
 - **ReLU (Rectified Linear Unit):** Speeds up training and reduces vanishing gradient issues.
 - **Sigmoid and Tanh:** Used in shallow networks or specific layers. Without activation functions, a neural network behaves like a linear model.
- 4. Weight Initialization:** Poor initialization can cause slow convergence or gradient issues. **Xavier initialization** (for sigmoid/tanh) and **He initialization** (for ReLU) help maintain stable gradients throughout the network.
- 5. Regularization Techniques:** Prevents overfitting and improves generalization to unseen data.
 - **Dropout:** Randomly deactivates neurons during training.
 - **L1/L2 Regularization:** Adds penalty terms to the loss to discourage large weights.
 - **Early stopping:** Stops training once validation accuracy stops improving.
- 6. Optimization Algorithms:** adjust weights based on gradients of loss function.
 - **SGD (Stochastic Gradient Descent):** Simple, efficient for large datasets.
 - **Adam (Adaptive Moment Estimation):** Combines momentum and adaptive learning rate.
- 7. Batch Normalization:** Normalizes outputs of layers to reduce internal covariate shift. Helps in **faster training, higher learning rates, better generalization**.
- 8. Modular and Reusable Design:** Models built as reusable components or layers. Encourages rapid experimentation and testing.

TensorFlow: open-source library, developed by **Google Brain Team**. A comprehensive ecosystem of tools for building, deploying models.

- 1. Open-Source and Multi-Platform Support:** This flexibility makes it suitable for both academic and commercial use.
- 2. Computational Graph Architecture:** uses **data flow graphs** to represent computation. Each **node** in the graph represents an operation (e.g., addition, multiplication), and **edges** represent tensors (multi-dimensional arrays). This approach allows efficient optimization and parallel computation across devices.
- 3. Eager Execution:** a flexible mode where operations are executed immediately, allows for easier debugging, step-by-step evaluation, and dynamic model building. Ideal for beginners and research-based applications.

- 4. Keras – High-Level API:** a simple and intuitive API to build neural networks quickly.
- 5. TensorBoard – Visualization Tool:** visualize model training in real-time. helps monitor loss and accuracy, view the computation graph, and debug gradients or parameters. This improves model interpretability and optimization.

6. Deployment Tools – Lite, JS, Serving:

- **TensorFlow Lite** for mobile and embedded devices.
- **TensorFlow.js** for running models in the browser using JavaScript.
- **TensorFlow Serving** for deploying models in production environments (e.g., web services).

7. GPU/TPU Support and Scalability:

- TensorFlow supports parallel training on **GPUs** and **TPUs (Tensor Processing Units)**.
- It also allows **distributed training** across multiple machines, making it highly scalable for large datasets and deep architectures.

8. Strong Ecosystem and Community:

- **TF Datasets** for data loading.
- **TF Hub** for reusable model components.
- **TF Model Garden** for pre-trained models.

Significance in Deep Learning: TensorFlow simplifies deep learning development through high-level APIs and modular design. It supports **end-to-end workflows** from data preprocessing, model design, training, evaluation, and deployment. Used in a variety of fields such as: **Healthcare** (e.g., cancer detection), **Finance** (e.g., fraud detection), **NLP** (e.g., chatbots, machine translation), **Computer Vision** (e.g., facial recognition, autonomous driving).

Role of Deep Learning in AlphaGo's Success:

- **AlphaGo:** an AI program developed by DeepMind to play the complex board game Go.
- Deep learning was critical in two parts of AlphaGo's architecture:
 - **Policy Network:** Deep neural networks trained to predict the next move by learning from expert human games and self-play.
 - **Value Network:** Estimates the probability of winning from a given game state.
- The networks enabled AlphaGo to evaluate an enormous number of possible board positions efficiently.
- Combined with **Monte Carlo Tree Search (MCTS)**, deep learning helped AlphaGo outperform traditional Go programs and defeated world champions.
- This success demonstrated deep learning's power in mastering tasks previously thought too complex for AI.

real-world applications of Deep Learning

- 1. Image and Video Recognition:** facial recognition systems (e.g., unlocking phones), medical imaging for detecting diseases such as cancer or diabetic retinopathy, autonomous vehicles for object detection and scene understanding.
- 2. Natural Language Processing (NLP):** virtual assistants like Siri, Alexa, and Google Assistant, machine translation services such as Google Translate, supports Chatbots and sentiment analysis in customer service.
- 3. Speech Recognition:** spoken language to text (e.g., dictation software), voice-controlled devices and transcription services, real-time language translation.
- 4. Recommendation Systems:** recommendations on platforms like Netflix, Amazon, and YouTube, Analyses user behaviour and preferences to suggest relevant content or products.
- 5. Autonomous Vehicles:** Enables self-driving cars to perceive the environment using cameras and sensors, processes complex data for navigation, obstacle detection, and decision making.

Aspect	Machine Learning (ML)	Deep Learning (DL)
Definition	A method of teaching computers to learn from data using algorithms like decision trees, SVM, etc	A specialized subset of ML that uses deep neural networks with many layers to learn from large datasets
Feature Engineering	Requires manual feature extraction and selection by domain experts	Automatically extracts features from raw data using multiple layers
Data Requirements	Works well with smaller to medium-sized datasets	Requires large amounts of data to perform effectively
Computational Power	Less computationally intensive; can run on CPUs	Requires high computational power, often GPUs or TPUs
Performance	Effective for simple to moderately complex problems	Excels at solving complex problems like image, speech recognition
Interpretability	More interpretable and transparent models	"black box" due to complex internal workings
Applications	Spam detection, credit scoring, customer churn prediction	Image recognition, natural language processing, autonomous driving